APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 2: Ensemble Models & Multiclass Decomposition

18 de octubre de 2026

Índice general

1 Apuntes del Tema 2: Ensemble Models & Multiclass Decomposition **2**

1.1 1. Motivación y Métodos de Ensamble (*Ensembles*) 2

1.1.1 1.1 Filosofía del Ensamble 2

1.2 2. Bagging y Random Forest 2

1.2.1 2.1 Bagging (*Bootstrap Aggregating*) 2

1.2.2 2.2 Random Forest 2

1.3 3. Boosting: AdaBoost y Gradient Boosting (XGBoost) 3

1.3.1 3.1 AdaBoost (*Adaptive Boosting*) 3

1.3.2 3.2 Gradient Boosting y XGBoost 3

1.4 4. Descomposición Multiclase: OVA y OVO 4

1.5 5. Práctica y Aplicación en Código (Scikit-Learn) 4

1 Apuntes del Tema 2: Ensemble Models & Multiclass Decomposition

1.1 1. Motivación y Métodos de Ensamble (*Ensembles*)

1.1.1 1.1 Filosofía del Ensamble

- **Limitaciones de un Modelo Individual:**
- Un modelo demasiado simple puede subajustar (*underfit*), mostrando un alto sesgo (*bias*) y bajo rendimiento.
- Un modelo excesivamente complejo puede sobreajustar (*overfit*), memorizando el ruido de los datos de entrenamiento y mostrando alta varianza (*variance*).
- Algunos modelos (como los árboles de decisión no podados) son inestables: pequeñas variaciones en los datos de entrenamiento generan cambios drásticos en la estructura del árbol.
- **Idea Clave:** Combinar múltiples estimadores débiles (*weak learners*) para construir un estimador fuerte (*strong learner*) cuya predicción colectiva sea más precisa, estable y robusta que cualquiera de los modelos individuales por separado.
- **Diversidad:** Para que un ensamble funcione eficazmente, los estimadores individuales deben cometer errores de manera independiente o desacoplada (diversidad de hipótesis).

1.2 2. Bagging y Random Forest

1.2.1 2.1 Bagging (*Bootstrap Aggregating*)

- **Objetivo:** Reducir la **varianza** de modelos inestables y con alta varianza (ej. árboles de decisión profundos) sin incrementar su sesgo.
- **Mecanismo Paso a Paso:**

1. **Muestreo Bootstrap:** Dado un dataset $\mathcal{D}$ de N muestras, se generan B subconjuntos $\mathcal{D}_b$ extrayendo N muestras con reemplazo de $\mathcal{D}$.
2. **Entrenamiento Independiente:** Se entrena un estimador $h_b(x)$ de forma completamente independiente sobre cada subconjunto $\mathcal{D}_b$.
3. **Agregación:**

- **Clasificación:** Voto mayoritario:

```
$$\{}hat{y} = \{}arg\{}max_{c} \{}sum_{b=1}^{B} \{}mathbb{I}(h_b(x) = c)$$
```

- **Regresión:** Promedio simple:

```
$$\{}hat{y} = \{}frac{1}{B} \{}sum_{b=1}^{B} h_b(x)$$
```

1.2.2 2.2 Random Forest

- **Mejora sobre Bagging:** En el Bagging estándar, si existe una característica dominante, todos los árboles la elegirán en la raíz, resultando en un conjunto de árboles altamente correlacionados.

- **Decorrelación de Árboles:** Random Forest añade una segunda fuente de aleatoriedad: en cada división (*split*) de cada árbol, se selecciona al azar un subconjunto reducido de m características de las d totales disponibles ($m \ll d$).
- Para clasificación: típicamente $m = \sqrt{d}$.
- Para regresión: típicamente $m = \frac{d}{3}$.
- **Evaluación Out-Of-Bag (OOB):** Dado que cada muestra *bootstrap* deja fuera aproximadamente un 36.8% de los datos ($1 - 1/e \approx 0.632$ incluidos), estas muestras "no vistas" sirven como conjunto de validación implícito (*OOB score*) sin requerir un split de validación adicional.

1.3 3. Boosting: AdaBoost y Gradient Boosting (XGBoost)

1.3.1 3.1 AdaBoost (*Adaptive Boosting*)

- **Mecanismo:** Entrena estimadores de forma **secuencial**. Mantiene una distribución de pesos $D_t(i)$ sobre las muestras del dataset.
- **Algoritmo Paso a Paso:**

1. Inicializa los pesos de las muestras uniformemente: $D_1(i) = 1/N$.
2. Para cada iteración $t = 1, \ldots, T$:

- Entrena un estimador débil $h_t(x)$ minimizando el error ponderado:

```
$$\{}epsilon_t = \{}sum_{i: y_i \{}neq h_t(x_i)} D_t(i)$$
```

- Calcula la importancia/confianza del estimador en la decisión final:

```
$$\{}alpha_t = \{}frac{1}{2} \{}ln\{}left(\{}frac{1 - \{}epsilon_t}{\{}epsilon_t}\{}right)$
```

- Actualiza los pesos de las muestras incrementando la importancia de las mal clasificadas:

```
$$D_{t+1}(i) \{}propto D_t(i) \{}cdot \{}exp\{}left(-\{}alpha_t y_i h_t(x_i)\{}right)$$
```

- Renormaliza D_{t+1} para que forme una distribución válida que sume 1.

1. **Predicción Final:** Voto ponderado:

```
$$\{}hat{y} = \{}text{signo}\{}left(\{}sum_{t=1}^{T} \{}alpha_t h_t(x)\{}right)$$
```

1.3.2 3.2 Gradient Boosting y XGBoost

- **Concepto Clave:** En lugar de reponderar muestras, cada nuevo árbol $h_t(x)$ se ajusta directamente para predecir los **residuos** (o el gradiente negativo de la función de pérdida) del ensamble acumulado $F_{t-1}(x)$.
- **Ajuste Aditivo:**

$$F_t(x) = F_{t-1}(x) + \nu h_t(x)$$

donde $\nu \in (0, 1]$ es el parámetro de tasa de aprendizaje (*shrinkage*) que regula la contribución de cada nuevo árbol para evitar el sobreajuste.

- **Residuos para Error Cuadrático Medio (MSE):**

$$r_{i,t-1} = -\left[\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{t-1}} = y_i - F_{t-1}(x_i)$$

- **XGBoost (*Extreme Gradient Boosting*):** Extensión altamente eficiente que incluye:
- Expansión de Taylor de segundo orden (utiliza tanto el gradiente de primer orden g_i como el Hessiano h_i).
- Regularización L_1 (α) y L_2 (λ) sobre las hojas del árbol.
- Manejo automático e intrínseco de valores faltantes (*sparse-aware split finding*).

1.4 4. Descomposición Multiclase: OVA y OVO

Cuando un modelo de clasificación binario puro (ej. Regresión Logística, SVM) debe aplicarse a un problema de K clases, se emplean estrategias de descomposición:

Estrategia	Cantidad de Clasificadores	Configuración del Entrenamiento	Mecanismo de Predicción
One-vs-All (OVA) *(One-vs-Rest)*	K clasificadores	El clasificador k se entrena con la clase k como positiva (+1) y todas las demás como negativas (-1).	Se asigna la clase cuyo clasificador entregue la mayor probabilidad o puntuación (*logit*): $\hat{y} = \arg\max_k f_k(x)$.
One-vs-One (OVO)	$\frac{K(K-1)}{2}$ clasificadores	Se entrena un clasificador para cada par único de clases (c_i, c_j) ignorando el resto.	Cada sub-clasificador emite un voto por su clase preferida. Gana la clase con **mayor número de votos acumulados**.

1.5 5. Práctica y Aplicación en Código (Scikit-Learn)

```
1
2 import numpy as np
3
4 from sklearn.datasets import make_classification
5
6 from sklearn.model_selection import train_test_split
7
8 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier,
    AdaBoostClassifier
```

```
9
10 from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
11
12 from sklearn.svm import SVC
13
14 from sklearn.metrics import accuracy_score
15
16
17
18 # ------------------------------------------------------------------------
19
20 # 1. Generación de Dataset Sintético Multiclase (K = 4 clases)
21
22 # ------------------------------------------------------------------------
23
24 X, y = make_classification(
25
26     n_samples=1000,
27
28     n_features=20,
29
30     n_informative=12,
31
32     n_classes=4,
33
34     random_state=42
35
36 )
37
38
39
40 X_train, X_test, y_train, y_test = train_test_split(
41
42     X, y, test_size=0.2, random_state=42, stratify=y
43
44 )
45
46
47
48 # ------------------------------------------------------------------------
49
50 # 2. Comparativa de Ensembles (Random Forest, AdaBoost, Gradient Boosting)
51
52 # ------------------------------------------------------------------------
53
54 def train_and_evaluate_ensembles(X_tr, y_tr, X_te, y_te):
55
56     models = {
57
58         "Random Forest": RandomForestClassifier(
59
60             n_estimators=100, max_features="sqrt", random_state=42
61
62         ),
63
64         "AdaBoost": AdaBoostClassifier(
65
66             n_estimators=50, learning_rate=0.1, random_state=42
```

```
        ),
        "Gradient Boosting": GradientBoostingClassifier(
            n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42
        )
    }

    print("--- Evaluación de Métodos de Ensamble ---")
    for name, model in models.items():
        model.fit(X_tr, y_tr)
        preds = model.predict(X_te)
        acc = accuracy_score(y_te, preds)
        print(f"Modelo: {name:<20} | Test Accuracy: {acc:.4f}")


# ------------------------------------------------------------------------
# 3. Estrategias de Descomposición Multiclase (OVA vs OVO)
# ------------------------------------------------------------------------
def train_multiclass_decomposition(X_tr, y_tr, X_te, y_te):
    # Clasificador base binario
    base_binary_clf = SVC(kernel='linear', C=1.0, random_state=42)

    # 1. One-vs-All (OVA / OvR)
    ova_model = OneVsRestClassifier(base_binary_clf)
    ova_model.fit(X_tr, y_tr)
    ova_preds = ova_model.predict(X_te)

    # 2. One-vs-One (OVO)
    ovo_model = OneVsOneClassifier(base_binary_clf)
    ovo_model.fit(X_tr, y_tr)
    ovo_preds = ovo_model.predict(X_te)
```

```
125
126
127
128     print("\n--- Estrategias de Descomposición Multiclase ---")
129
130     print(f"Número de estimadores OVA (K=4)    : {len(ova_model.estimators_)}")
131
132     print(f"Accuracy OVA                       : {accuracy_score(y_te, ova_preds):.4f}
        ")
133
134
135
136     print(f"Número de estimadores OVO (K*(K-1)/2): {len(ovo_model.estimators_)}")
137
138     print(f"Accuracy OVO                       : {accuracy_score(y_te, ovo_preds):.4f}
        ")
139
140
141
142 if __name__ == "__main__":
143
144     train_and_evaluate_ensembles(X_train, y_train, X_test, y_test)
145
146     train_multiclass_decomposition(X_train, y_train, X_test, y_test)
```